プログラミングワークショップ２

青森大学ソフトウェア情報学部

橋本恭能

haship@aomori-u.ac.jp

講義資料について

- この講義で使う資料やプリントは以下のWebページから閲覧できます
- ブラウザを開き、以下のアドレスを入力してください

URL http://haship.github.io

- Webページ上部の【講義資料】をクリック
- 表から【プログラミングワークショップ２】をクリック
- 目次ページをブックマークする

今後の予定

回	内容
~~1~~	~~ソフトウェア開発1~~
~~2~~	~~ソフトウェア開発2~~
~~3~~	~~ソフトウェア開発3~~
~~4~~	~~ソフトウェア開発4~~
~~5~~	~~中間発表1(テーマ発表)~~
~~6~~	~~ソフトウェア開発5~~
~~7~~	~~ソフトウェア開発6~~
~~8~~	~~ソフトウェア開発7~~

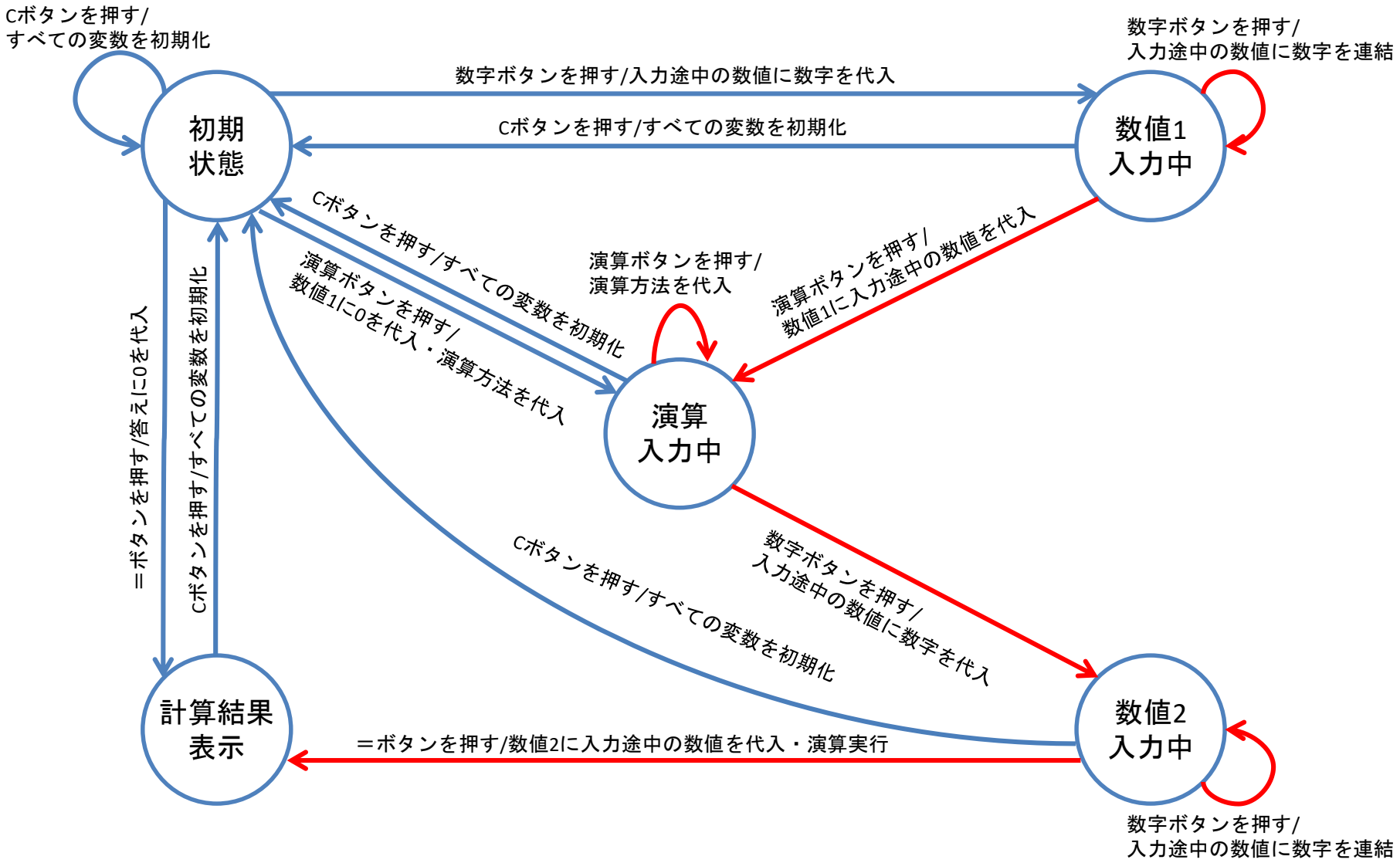
回	内容
~~9~~	~~ソフトウェア開発8~~
~~10~~	~~中間発表2(開発状況)~~
~~11~~	~~中間発表2(開発状況)つづき~~
~~12~~	~~ソフトウェア開発9~~
13	ソフトウェア開発10
14	ソフトウェア開発11(補講)
15	最終発表(補講)

- ソフトウェア開発：12回
- 中間発表：2回（テーマ発表、開発状況）
- 最終発表：1回

状態遷移表

	初期状態	数値1 入力中	演算 入力中	数値2 入力中	結果 表示中
数字ボタンを押す	入力途中の数値に数字を代入	入力途中の数値に数字を連結	入力途中の数値に数字を代入	入力途中の数値に数字を連結	
	数値1入力中	数値1入力中	数値2入力中	数値2入力中	
演算ボタンを押す	数値1に0を代入 演算方法を代入	数値1に 入力途中の数値を代入	演算方法を代入		
	演算入力中	演算入力中	演算入力中		
＝ボタンを押す	答えに0を代入			数値2に入力途中の数値を代入・演算実行	
	結果表示中			結果表示中	
Cボタンを押す	すべての変数を初期化	すべての変数を初期化	すべての変数を初期化	すべての変数を初期化	すべての変数を初期化
	初期状態	初期状態	初期状態	初期状態	初期状態

状態遷移図


Cボタンを押す/
すべての変数を初期化
初期
状態
数字ボタンを押す/入力途中の数値に数字を代入
Cボタンを押す/すべての変数を初期化
数値1
入力中
数字ボタンを押す/
入力途中の数値に数字を連結
Cボタンを押す/すべての変数を初期化
演算ボタンを押す/
演算方法を代入
演算ボタンを押す/
数値1に入力途中の数値を代入
演算ボタンを押す/
数値1に0を代入・演算方法を代入
演算
入力中
＝ボタンを押す/答えに0を代入
Cボタンを押す/すべての変数を初期化
Cボタンを押す/すべての変数を初期化
数字ボタンを押す/
入力途中の数値に数字を代入
計算結果
表示
＝ボタンを押す/数値2に入力途中の数値を代入・演算実行
数値2
入力中
数字ボタンを押す/
入力途中の数値に数字を連結

メインタブ

```
//メインタブ sample03

Calc calc;

void setup() {
  size(400, 475);
  calc = new Calc();
}

void draw() {

  calc.run();

}
```

Buttonクラスにメソッド追加

- boolean isClicked()
  - ボタンを押していない状態から押した状態への変化があったらtrueを返す
  - 以前ボタンを押したかどうかと現在ボタンを押したかどうかの、2つの値を使う
- String getName()
  - ボタンの名前 name を返す

Buttonクラス①

```
//Buttonクラス
class Button {
  int x, y, w, h;          //x座標、y座標、幅、高さ
  String name;             //ボタンの名前
  boolean wasPressed;      //以前にボタンを押したかどうかを保存する
  int buttonColor = color(255);          //通常時のボタンの色
  int hoverColor  = color(0, 255, 255);  //カーソルがボタン上にある時の色
  int pushColor   = color(255, 255, 0);  //押したときの色
  int textColor   = color(0);            //テキストの色

  //コンストラクタ
  Button(int _x, int _y, int _w, int _h, String _name) {
    x = _x;
    y = _y;
    w = _w;
    h = _h;
    name = _name;
  }
```

Buttonクラス②

```
//カーソルがボタン上にあるかどうか
boolean isHover() {
  if ((mouseX > x) && (mouseX < x+w) &&
    (mouseY > y) && (mouseY < y+h)) {
    return true;
  } else {
    return false;
  }
}

//ボタンを押したかどうか
boolean isPressed() {
  if (isHover() && mousePressed) {
    return true;
  } else {
    return false;
  }
}
```

Buttonクラス③

```
//ボタンをクリックしたかどうか
boolean isClicked() {
  boolean isPressed = isPressed();  //現在の押下状態を取得
  boolean res;

  if (!wasPressed && isPressed) {  //前に押していない && 今押している
    res = true;
  } else {
    res = false;
  }
  wasPressed = isPressed;  //更新
  return res;
}

//ボタンの名前を返す
String getName() {
  return name;
}
```

Buttonクラス④

```
  //ボタンを表示する
  void display() {
    //ボタンを描く
    if (isPressed()) {            //ボタンが押された場合
      fill(pushColor);            //押したときの色を指定
    } else if (isHover()) {       //カーソルがボタン上にある場合
      fill(hoverColor);           //カーソルがボタン上にある時の色を指定
    } else {                      //それ以外
      fill(buttonColor);          //通常の色を指定
    }
    rect(x, y, w, h);             //長方形を表示
    //ボタンの上に文字を書く
    textAlign(CENTER, CENTER);  //テキストの配置を上下中央、左右中央に指定
    textSize(32);               //テキストサイズを指定
    fill(textColor);            //テキストの色を指定
    text(name, x, y, w, h);     //テキストを長方形内に書く
  }
}
```

Screenクラス①

```
//Screenクラス

class Screen {

  int x, y, w, h;  //x座標、y座標、幅、高さ
  int screenColor = color(0);    //画面の色
  int textColor   = color(255);  //文字の色

  //コンストラクタ
  Screen(int _x, int _y, int _w, int _h){
    x = _x;
    y = _y;
    w = _w;
    h = _h;
  }
```

Screenクラス②

```
//画面を表示する
  void display(String s){

    fill(screenColor);  //画面の色を指定
    rect(x, y, w, h);   //長方形を描く

    textAlign(RIGHT, CENTER);  //文字の配置を右寄せ、上下中央寄せに指定
    textSize(48);              //テキストサイズ指定
    fill(textColor);           //テキストの色を指定
    text(s, x+10, y+10, w-20, h-20);  //画面外枠の10pt内側に文字を書く
  }

}
```

Calcクラス変更

- フィールド追加
  - 数値1　　float num1;
  - 数値2　　float num2;
  - 入力途中の数値
    String input;
  - 演算方法　String ope;
  - 答え　　float answer;
  - 現在の状態
    State state;

- メソッド追加・変更
  - 電卓を表示する（変更）
    void display(String str)
  - 実行する
    void run()
  - 押したボタンの名前を返す
    String getButtonName()

Calcクラス①

```
//状態
enum State {
    INIT,        //初期状態
    INPUT_NUM1,  //数値1入力中
    SELECT_OP,   //演算入力中
    INPUT_NUM2,  //数値2入力中
    RESULT       //結果表示中
};
//Calcクラス
class Calc {
  Screen screen; //画面
  Button[] btn;  //ボタン(配列)
  float num1;    //数値1
  float num2;    //数値2
  String input;  //入力途中の数値
  String ope;    //演算方法
  float answer;  //答え
  State state;   //現在の状態
```

Calcクラス②

```
//コンストラクタ
Calc() {
  num1 = 0.0;
  num2 = 0.0;
  answer = 0.0;
  input = "0";
  ope = "";
  state = State.INIT;

  screen = new Screen(0, 0, 400, 100);
  btn = new Button[17];
  btn[0]  = new Button(0, 400, 200, 75, "0");
  btn[1]  = new Button(0, 325, 100, 75, "1");
  btn[2]  = new Button(100, 325, 100, 75, "2");
  btn[3]  = new Button(200, 325, 100, 75, "3");
  btn[4]  = new Button(0, 250, 100, 75, "4");
  btn[5]  = new Button(100, 250, 100, 75, "5");
  btn[6]  = new Button(200, 250, 100, 75, "6");
```

Calcクラス③

```
    btn[7]  = new Button(0, 175, 100, 75, "7");
    btn[8]  = new Button(100, 175, 100, 75, "8");
    btn[9]  = new Button(200, 175, 100, 75, "9");
    btn[10] = new Button(200, 400, 100, 75, ".");
    btn[11] = new Button(300, 325, 100, 150, "=");
    btn[12] = new Button(300, 175, 100, 150, "+");
    btn[13] = new Button(300, 100, 100, 75, "-");
    btn[14] = new Button(200, 100, 100, 75, "*");
    btn[15] = new Button(100, 100, 100, 75, "/");
    btn[16] = new Button(0, 100, 100, 75, "C");
  }

  //電卓を表示する
  void display(String str) {
    //画面を表示する
    screen.display(str);
    //ボタンを表示する
    for (int i=0; i<btn.length; i++) {
```

Calcクラス④

```
    btn[i].display();
  }
}

//実行する
void run() {
  switch(state) {
  //初期状態の場合
  case INIT:
    display(input);
    break;
 //数値1入力中の場合
  case INPUT_NUM1:
    break;
 //演算入力中の場合
  case SELECT_OP:
    break;
```

Calcクラス⑤

```
    //数値2入力中の場合
    case INPUT_NUM2:
      break;
    //結果表示中の場合
    case RESULT:
      break;
    //その他
    default:
      break;
    }
  }
  //押したボタンの名前を返す
  String getButtonName() {
    String res = "";

    for (int i=0; i<btn.length; i++) { //すべてのボタンで繰り返す
      if (btn[i].isClicked()) {  //i番目のボタンをクリックしたら
        res = btn[i].getName();  //そのボタンの名前を代入する
```

Calcクラス⑥

```
        break;                          //繰り返しから抜ける
      }
    }

    return res;
  }
}
```

初期状態の処理

- 必ず実行する処理
  - 各変数の初期化
  - 入力途中の数値を表示する
    （画面・ボタン表示含む）
- イベント発生時の処理
  - 状態遷移表を参照
  - イベントが起きたかどうかを確認するメソッドが必要

イベントを確認する

- 数字ボタンを押したかどうか

```
boolean isClickedNumButton(String b)
```

- 演算ボタンを押したかどうか

```
boolean isClickedOpeButton(String b)
```

- ＝ボタンを押したかどうか

```
boolean isClickedEqButton(String b)
```

- Cボタンを押したかどうか

```
boolean isClickedClearButton(String b)
```

メソッド追加

```
// Calcクラス
class Calc {

  //途中省略

  //数字ボタンを押したかどうか
  boolean isClickedNumButton(String s) {}

  //演算ボタンを押したかどうか
  boolean isClickedOpeButton(String s) {}

  //=ボタンを押したかどうか
  boolean isClickedEqButton(String s) {}

  //Cボタンを押したかどうか
  boolean isClickedClearButton(String s) {}

}
```

数字ボタンを押したかどうか

```
//数字ボタンを押したかどうか
boolean isClickedNumButton(String s) {
  //０～９のどれかのボタンを押したらtrue，それ以外はfalseを返す
  if (s == "0" || s == "1" || s == "2" || s == "3" ||
    s == "4" || s == "5" || s == "6" || s == "7" ||
    s == "8" || s == "9") {
    return true;
  } else {
    return false;
  }
}
```

ほかのメソッド

```
//演算ボタンを押したかどうか
  boolean isClickedOpeButton(String s) {
  // ”+”, ”-”, ”*”, ”/”のどれかを押したらtrue、それ以外はfalseを返す
  }

  //=ボタンを押したかどうか
  boolean isClickedEqButton(String s) {
  // ”=” を押したらtrue、それ以外はfalseを返す
  }

  //Cボタンを押したかどうか
  boolean isClickedClearButton(String s) {
  // ”C” を押したらtrue、それ以外はfalseを返す
  }
```

コメントをもとに
３つのメソッドを実装すること

run()

```
//実行する
void run() {

  String b = getButtonName();     //押したボタンの名前を取得、代入する

  switch(state) {
  //初期状態の場合
  case INIT:

    //必ず実行する処理
    num1 = 0.0;           //数値1を0に初期化
    num2 = 0.0;           //数値2を0に初期化
    input = “0”;          //入力途中の数値を0に初期化
    ope = “”;             //演算方法を初期化
    display(input);       //入力途中の数値を画面に表示
```

run()

```
//イベントごとの処理
if (isClickedNumButton(b)) {  //数字ボタンを押した場合
  input += b;                 //押した数値を入力途中の数値に連結する
  state = State.INPUT_NUM1;   //現在の状態を数値1入力中に遷移
}
else if (isClickedOpeButton(b)) {      //演算ボタンを押した場合
  num1 = 0.0;                 //数値1に0を代入
  ope = b;                    //演算方法に入力した演算を代入
  state = State.SELECT_OP;    //現在の状態を演算入力中に遷移
}
else if (isClickedEqButton(b)) {  //=ボタンを押した場合
  answer = 0.0;               //答えに0を代入
  state = State.RESULT;       //現在の状態を結果表示中に遷移
}
else if (isClickedClearButton(b)) {  //Cボタンを押した場合
  state = State.INIT;         //現在の状態を初期状態に遷移
}
break;
```